

Advanced Numpy: Comprehensive Lecture Notes

Source: CampusX Data Science Mentorship Program (DSMP) - Session 14

Instructor: Nitish

Table of Contents

1. Numpy Arrays vs. Python Lists
2. Advanced Indexing (Fancy & Boolean)
3. Broadcasting (Rules and Examples)
4. Implementing Mathematical Formulas
5. Handling Missing Values (np.nan)
6. Plotting Graphs with Matplotlib & Numpy

1. Numpy Arrays vs. Python Lists [00:07:00]

Numpy was introduced primarily because Python's built-in data types (like lists) were too slow and consumed too much memory for large-scale data manipulation. We evaluate the differences on three metrics:

A. Speed of Execution

Numpy is significantly faster (approx. 50 times faster) than Python lists.

- **Why?** Python lists are dynamic and referential (they store memory addresses of items, not the items themselves). Numpy arrays are C-type arrays; they are static and store items directly in contiguous memory blocks.
- **Demonstration:** Adding two lists of 1 crore (10 million) items element-wise using a for loop in Python takes ~3.26 seconds. Doing the same via $c = a + b$ in Numpy takes roughly ~0.06 seconds.

B. Memory Occupation

Numpy consumes far less memory than Python lists.

- A standard Python list with 1 crore elements occupies a massive amount of memory (around 80+ MB) because of how Python handles integer objects and references.
- A Numpy array of the same size uses much less. Furthermore, Numpy provides **flexibility in data types**. By default, Numpy might use int64. If you know your numbers are small, you can specify dtype=np.int32, np.int16, or np.int8.

```
import numpy as np
import sys
```

```
# Changing dtype reduces memory drastically
a = np.arange(10000000, dtype=np.int32)
b = np.arange(10000000, dtype=np.int8) # Takes even less space
```

C. Convenience

Element-wise operations in Python lists require writing complex for loops. In Numpy, you can perform element-wise addition simply by writing `a + b`.

2. Advanced Indexing [00:23:00]

Beyond standard indexing (e.g., `a[1:3, 1:3]`), Numpy offers two advanced ways to extract data from arrays.

A. Fancy Indexing

Used when you need to fetch specific, non-sequential rows or columns where no clear slice pattern exists.

- **Concept:** Pass a list of the exact indices you want inside the square brackets.

```
# Create a sample array
a = np.arange(24).reshape(6, 4)
```

```
# I want specifically the 1st, 3rd, 4th, and 6th rows (Indices: 0, 2, 3, 5)
fancy_rows = a[[0, 2, 3, 5]]
```

```
# I want the 1st, 2nd, and 3rd columns across all rows (Indices: 0, 1, 2)
fancy_cols = a[:, [0, 1, 2]]
```

B. Boolean Indexing

Super important for Data Analysis (heavily used in Pandas later). It involves filtering data based on conditions.

```
# Create a random array of 24 items between 1 and 100
a = np.random.randint(1, 100, 24).reshape(6, 4)
```

```
# 1. Find all numbers strictly greater than 50
mask = a > 50 # Creates a boolean mask (Array of True/False)
result = a[mask] # OR simply: a[a > 50]
```

```
# 2. Find all EVEN numbers
```

```
even_nums = a[a % 2 == 0]
```

3. Multiple Conditions: Numbers > 50 AND Even

NOTE: Use bitwise operator '&' for boolean arrays, not the logical 'and'

```
result_multi = a[(a > 50) & (a % 2 == 0)]
```

4. Negation: Numbers NOT divisible by 7

Use the tilde (~) symbol to invert the boolean mask

```
not_div_7 = a[~(a % 7 == 0)]
```

3. Broadcasting [00:40:40]

Definition: Broadcasting describes how Numpy treats arrays with different shapes during arithmetic operations. It "broadcasts" (stretches) the smaller array across the larger one so that they have compatible shapes.

Broadcasting Rules:

Numpy compares dimensions element-wise, starting from the trailing dimensions (right to left). Two dimensions are compatible if:

1. **Rule 1:** Make dimensions identical. If dimensions differ, prepend 1 to the shape of the smaller dimension array until they match. (e.g., *shape (3,) becomes (1, 3)*).
2. **Rule 2:** Match the sizes. If the sizes of a dimension do not match, the array with size 1 in that dimension is stretched to match the other array.
3. **Rule 3:** If in any dimension the sizes disagree and neither is equal to 1, an **Error** is thrown.

Practical Examples:

Example 1: Success

- Array A: (4, 3)
- Array B: (3,)
- *Step 1:* Pad B with 1: (1, 3)
- *Step 2:* Stretch B's 1 to 4: (4, 3)
- *Result:* Compatible! Broadcasting happens.

Example 2: Failure

- Array A: (3, 4)
- Array B: (4, 3)
- *Step 1:* Dimensions are already same (2D).
- *Step 2:* Check elements. Left side wants to stretch 3 to 4, but it's not a 1.
- *Result:* Error!

Example 3: Success

- Array A: (1, 3)
- Array B: (3, 1)
- Step 1: Stretch A's row to match B: (3, 3)
- Step 2: Stretch B's column to match A: (3, 3)
- Result: Compatible! Both become (3, 3).

4. Implementing Mathematical Formulas [01:04:00]

While Numpy has built-in functions like `np.sum()`, `np.sin()`, etc., in Machine Learning and Deep Learning, you often need to calculate complex mathematical functions. Because of Numpy's element-wise operations, creating these functions is just one line of code.

A. Sigmoid Function

Widely used in Logistic Regression and Deep Learning.

- **Formula:** $S(x) = \frac{1}{1+e^{-x}}$

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
# Generate an array
```

```
a = np.arange(10)
```

```
# Apply Sigmoid - It will calculate for all elements instantly without loops
```

```
result = sigmoid(a)
```

B. Mean Squared Error (MSE)

A famous Loss Function used in Linear Regression to calculate prediction errors.

- **Formula:** $MSE = \frac{1}{n} \sum (Y_{actual} - Y_{predicted})^2$

```
actual = np.random.randint(1, 50, 25)
```

```
predicted = np.random.randint(1, 50, 25)
```

```
def mse(actual, predicted):
```

```
    # Step 1: actual - predicted
```

```
    # Step 2: ** 2 (Square it)
```

```
    # Step 3: np.mean (Find the average)
```

```
    return np.mean((actual - predicted)**2)
```

```
error = mse(actual, predicted)
```

```
print("Mean Squared Error:", error)
```

5. Working with Missing Values [01:23:00]

In real-world data, missing values are highly common. In Numpy, missing values are represented as `np.nan` (Not a Number).

Note: `np.nan` is considered a distinct entity. You cannot check for it using standard equality (e.g., `x == np.nan` will not work as expected).

```
# Creating an array with missing values
```

```
a = np.array([1, 2, 3, np.nan, 5, np.nan])
```

```
# Finding missing values (Returns boolean array where True means NaN)
```

```
missing_mask = np.isnan(a)
```

```
# Filtering OUT the missing values
```

```
# Use the NOT operator (~) to keep only items that are NOT nan
```

```
clean_a = a[~np.isnan(a)]
```

6. Plotting Graphs [01:29:30]

Numpy, paired with `matplotlib.pyplot`, allows you to plot absolutely any mathematical function easily by generating X coordinates and formulating Y mathematically.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# 1. Plotting a straight line passing through origin (y = x)
```

```
x = np.linspace(-10, 10, 100) # 100 numbers between -10 and 10
```

```
y = x
```

```
plt.plot(x, y)
```

```
# 2. Plotting a Parabola (y = x^2)
```

```
x = np.linspace(-10, 10, 100)
```

```
y = x ** 2
```

```
plt.plot(x, y)
```

```
# 3. Plotting a Sine Wave (y = sin(x))
```

```
x = np.linspace(-10, 10, 100)
```

```
y = np.sin(x)
```

```
plt.plot(x, y)
```

```
# 4. Plotting  $y = x * \log(x)$   
x = np.linspace(1, 10, 100) # Started from 1 because log(0) is undefined/infinity  
y = x * np.log(x)  
plt.plot(x, y)
```

```
# 5. Plotting Sigmoid curve ( $y = 1 / 1 + e^{-x}$ )  
x = np.linspace(-10, 10, 100)  
y = 1 / (1 + np.exp(-x))  
plt.plot(x, y)
```